

UNIVERSIDADE DE BRASÍLIA

**Sistema de Vendas e Fidelidade
Cafeteria Geek “Byte & Brew”**

Trabalho Prático de Orientação a Objetos

Denilson Lima Santos — Matrícula: 242015460

Emanuele Victor Santos — Matrícula: 242015601

Victor Hugo Ribeiro de Almeida Coelho — Matrícula: 242015970

Disciplina: Orientação a Objetos

Brasília, 29 de junho de 2026

Sumário

1	Introdução	2
2	Classes, Objetos e Associações	2
3	Modificadores de Escopo (Estático vs. Dinâmico)	2
4	Herança Simples	3
5	Polimorfismo por Inclusão	3
6	Polimorfismo por Sobrescrita (<i>Override</i>)	4
7	Polimorfismo por Sobrecarga (<i>Overload</i>)	5
8	Polimorfismo por Coerção (Conversão de Tipos)	5
9	Interfaces	6
10	Exceções Customizadas	6
11	Encapsulamento	7
12	Modularidade (Pacotes)	7
13	Conclusão	8

1 Introdução

Este documento apresenta a documentação técnica do Sistema de Vendas e Fidelidade desenvolvido para a cafeteria temática geek “Byte & Brew”, elaborado como Trabalho Prático da disciplina de Orientação a Objetos.

O sistema gerencia três pilares principais do negócio: o cadastro de produtos (Comidas e Bebidas), o cadastro de clientes vinculados ao programa de fidelidade (Clientes Standard e VIP, identificados por XP) e o fluxo completo de vendas, incluindo controle de estoque, aplicação de descontos promocionais e resgate de pontos.

O código-fonte foi implementado em Java, organizado em quatro pacotes (`modelo`, `excecao`, `servico` e `app`), e aplica de forma deliberada os principais conceitos de Orientação a Objetos exigidos no enunciado: associações entre classes, herança, as quatro formas de polimorfismo (inclusão, sobrescrita, sobrecarga e coerção), interfaces, exceções customizadas e encapsulamento estrito.

As seções a seguir localizam, no código-fonte, cada um desses conceitos, com trechos de código e explicação de sua aplicação prática no contexto do sistema.

2 Classes, Objetos e Associações

Onde está: pacote `br.edu.cafeteria.modelo`

A classe `Pedido` possui uma associação com `Cliente` (um pedido pode ter zero ou um cliente vinculado) e uma associação de composição com `ItemPedido` (um pedido contém uma lista de itens). Por sua vez, `ItemPedido` está associado a `Produto` (cada item referencia um produto e uma quantidade).

```
public class Pedido {
    private List<ItemPedido> itens;
    private Cliente cliente;
    ...
}

public class ItemPedido {
    private Produto produto;
    private int quantidade;
    ...
}
```

Essa cadeia de associações (`Pedido` → `ItemPedido` → `Produto`, e `Pedido` → `Cliente`) reflete diretamente o fluxo de negócio: um pedido é composto por itens, e cada item se refere a um produto do cardápio.

3 Modificadores de Escopo (Estático vs. Dinâmico)

Onde está: `Pedido.java` e `ClienteVIP.java`

Atributo estático — gera o número sequencial automático dos pedidos, compartilhado entre todas as instâncias da classe:

```
public class Pedido {
    private static int contadorSequencial = 1;

    public Pedido() {
        this.id = contadorSequencial++;
        ...
    }
}
```

Constante estática — representa a taxa de conversão de pontos XP em desconto, usada no resgate de pontos do Cliente VIP:

```
public class ClienteVIP extends Cliente {
    private static final double TAXA_CONVERSAO_XP = 10.0;

    public void pagarComXP(double valorTotal) throws
        PontosInsuficientesException {
        double custoEmPontos = valorTotal * TAXA_CONVERSAO_XP;
        ...
    }
}
```

Há também a constante `PERCENTUAL_DESCONTO` em `DescontoEventoGeek`, usada para o desconto de 10% em bebidas durante o “Dia de Evento Geek”.

4 Herança Simples

Onde está: pacote `br.edu.cafeteria.modelo`

Comida e Bebida herdam de `Produto`:

```
public abstract class Produto { ... }
public class Comida extends Produto { ... }
public class Bebida extends Produto { ... }
```

`ClienteStandard` e `ClienteVIP` herdam de `Cliente`:

```
public abstract class Cliente { ... }
public class ClienteStandard extends Cliente { ... }
public class ClienteVIP extends Cliente { ... }
```

`Produto` e `Cliente` são classes abstratas: definem os atributos e comportamentos comuns a todas as subclasses, mas não fazem sentido sozinhas (não existe um “produto genérico” sem ser Comida ou Bebida).

5 Polimorfismo por Inclusão

Onde está: `Pedido.java`

A classe `Pedido` armazena uma lista de `ItemPedido`, e cada `ItemPedido` referencia um `Produto` — que pode ser, na prática, uma instância de `Comida` ou `Bebida`. O sistema processa essa lista de forma genérica, sem precisar saber o tipo concreto de cada produto:

```
public List<ItemPedido> getItens() { return itens; }

public double calcularTotal() {
    double total = 0;
    for (ItemPedido item : itens) {
        total += item.calcularSubtotal();
    }
    return total;
}
```

O mesmo padrão aparece em `DescontoEventoGeek.aplicarDesconto()`, que percorre a lista de itens de um pedido tratando-os genericamente como `Produto`, e só verifica o tipo concreto (`Bebida`) quando necessário para aplicar a regra de negócio.

6 Polimorfismo por Sobrescrita (*Override*)

Onde está: `Cliente.java`, `ClienteStandard.java`, `ClienteVIP.java`

O método `adicionarXP()` é declarado como abstrato na superclasse `Cliente` e sobrescrito em cada subclasse, refletindo a taxa de acúmulo de XP de cada categoria:

```
public abstract class Cliente {
    public abstract void adicionarXP(double valorCompra);
}

public class ClienteStandard extends Cliente {
    @Override
    public void adicionarXP(double valorCompra) {
        this.saldoXP += valorCompra * 1.0;
    }
}

public class ClienteVIP extends Cliente {
    @Override
    public void adicionarXP(double valorCompra) {
        this.saldoXP += valorCompra * 2.0;
    }
}
```

Quando o `Main` chama `cliente.adicionarXP(valorFinal)`, o método executado em tempo de execução depende do tipo real do objeto (`Standard` ou `VIP`) — esse é o polimorfismo de sobrescrita em ação.

7 Polimorfismo por Sobrecarga (*Overload*)

Onde está: `Pedido.java`

A classe `Pedido` define duas versões do método `adicionarItem`, com assinaturas diferentes:

```
public void adicionarItem(Produto p) throws
    EstoqueInsuficienteException {
    this.adicionarItem(p, 1);
}

public void adicionarItem(Produto p, int quantidade) throws
    EstoqueInsuficienteException {
    if (quantidade > p.getQuantidadeEstoque()) {
        throw new EstoqueInsuficienteException(...);
    }
    ItemPedido novoItem = new ItemPedido(p, quantidade);
    this.itens.add(novoItem);
    p.setQuantidadeEstoque(p.getQuantidadeEstoque() -
        quantidade);
}
```

A versão sem quantidade reaproveita a versão completa, adicionando 1 unidade por padrão — evitando duplicação de lógica.

8 Polimorfismo por Coerção (Conversão de Tipos)

Onde está: `Main.java`, método `novaVenda()`

Quando o cliente vinculado ao pedido é VIP, o sistema precisa acessar o método `pagarComXP()`, que existe apenas em `ClienteVIP` e não na superclasse `Cliente`. Isso exige um *down-cast* (conversão de tipo explícita), feito de forma consciente após uma verificação com `instanceof`:

```
Cliente cliente = pedido.getCliente();
if (cliente instanceof ClienteVIP) {
    ClienteVIP vip = (ClienteVIP) cliente; // coercao: Cliente
        -> ClienteVIP
    ...
    vip.pagarComXP(valorFinal);
}
```

Também ocorre coerção implícita (*widening*) em diversas operações numéricas do sistema, como na multiplicação de um `int` (quantidade) por um `double` (preço base) em `ItemPedido.calcularSubtotal()`:

```
public double calcularSubtotal() {
    return produto.getPrecoBase() * quantidade; // double * int
        -> double
}
```

9 Interfaces

Onde está: pacote `br.edu.cafeteria.servico`

A interface `Promocional` define o contrato para qualquer regra de desconto que o sistema venha a ter:

```
public interface Promocional {  
    void aplicarDesconto(Pedido pedido);  
}
```

A classe `DescontoEventoGeek` implementa essa interface, aplicando 10% de desconto sobre o valor das bebidas quando ativado o “Dia de Evento Geek”:

```
public class DescontoEventoGeek implements Promocional {  
    private static final double PERCENTUAL_DESCONTO = 0.10;  
  
    @Override  
    public void aplicarDesconto(Pedido pedido) {  
        double descontoTotal = 0;  
        for (ItemPedido item : pedido.getItems()) {  
            if (item.getProduto() instanceof Bebida) {  
                descontoTotal += item.calcularSubtotal() *  
                    PERCENTUAL_DESCONTO;  
            }  
        }  
        pedido.setDesconto(descontoTotal);  
    }  
}
```

O uso da interface permite que o `Main` acione a regra de desconto sem conhecer a implementação concreta, apenas o contrato:

```
Promocional promo = new DescontoEventoGeek();  
promo.aplicarDesconto(pedido);
```

Essa estrutura também deixa o sistema preparado para novas promoções no futuro (por exemplo, um desconto de aniversário), sem alterar o código existente — bastaria criar uma nova classe que implemente `Promocional`.

10 Exceções Customizadas

Onde estão: pacote `br.edu.cafeteria.excecao`

`EstoqueInsuficienteException`

Lançada em `Pedido.adicionarItem()` quando a quantidade solicitada excede o estoque disponível do produto:

```
if (quantidade > p.getQuantidadeEstoque()) {
```

```

        throw new EstoqueInsuficienteException(
            "Estoque insuficiente para o produto: " + p.getNome()
            + " (Disponível: " + p.getQuantidadeEstoque() + ")"
        );
    }

```

PontosInsuficientesException

Lançada em `ClienteVIP.pagarComXP()` quando o saldo de XP do cliente é insuficiente para cobrir o valor da compra em pontos:

```

if (this.saldoXP < custoEmPontos) {
    throw new PontosInsuficientesException(
        "Saldo de XP insuficiente para resgate. Necessario: "
        + custoEmPontos + " XP. Seu saldo atual: " + this.
        saldoXP + " XP."
    );
}

```

Justificativa: por que são *checked exceptions*

Ambas as exceções estendem `Exception` (não `RuntimeException`), tornando-as **checked exceptions**. Essa escolha foi deliberada:

- São situações de regra de negócio previsíveis e recuperáveis — não indicam um erro de programação, mas uma condição esperada do domínio (estoque pode acabar, saldo pode ser insuficiente).
- Ao serem *checked*, o compilador obriga quem chama `adicionarItem()` ou `pagarComXP()` a tratar o erro explicitamente (com `try/catch` ou propagando com `throws`), evitando que essas situações sejam ignoradas acidentalmente.
- No `Main`, isso se reflete diretamente: a chamada a `adicionarItem()` é envolvida em `try/catch`, permitindo que o atendente seja informado do problema e continue o atendimento, em vez de o programa simplesmente falhar.

11 Encapsulamento

Todos os atributos das classes do pacote `modelo` são declarados como `private` (ou `protected`, quando precisam ser acessados diretamente pelas subclasses, como `saldoXP` em `Cliente`). O acesso externo ocorre exclusivamente por meio de métodos `get/set` ou métodos de comportamento (como `adicionarXP()`, `adicionarItem()`, `pagarComXP()`), nunca por exposição direta dos atributos.

12 Modularidade (Pacotes)

O projeto está organizado em quatro pacotes, conforme exigido:

Pacote	Conteúdo
<code>br.edu.cafeteria.modelo</code>	Classes de negócio: <code>Produto</code> , <code>Comida</code> , <code>Bebida</code> , <code>Cliente</code> , <code>ClienteStandard</code> , <code>ClienteVIP</code> , <code>Pedido</code> , <code>ItemPedido</code>
<code>br.edu.cafeteria.excecao</code>	<code>EstoqueInsuficienteException</code> , <code>PontosInsuficientesException</code>
<code>br.edu.cafeteria.servico</code>	<code>Promocional</code> (interface), <code>DescontoEventoGeek</code> (implementação)
<code>br.edu.cafeteria.app</code>	<code>Main</code> (classe principal com o método <code>main</code>)

13 Conclusão

O desenvolvimento do Sistema de Vendas e Fidelidade da cafeteria “Byte & Brew” permitiu aplicar, em um cenário prático e coeso, os principais pilares da Orientação a Objetos estudados na disciplina. A modelagem das classes `Produto`, `Cliente` e `Pedido` — e de suas respectivas especializações e associações — exigiu decisões de projeto que vão além da sintaxe da linguagem, como a escolha entre herança e composição, a definição de quais atributos deveriam ser estáticos, e a forma mais adequada de tratar situações excepcionais do domínio de negócio.

A exigência de implementar as quatro formas de polimorfismo (inclusão, sobrescrita, sobrecarga e coerção) em um único sistema coerente reforçou a compreensão de que esses conceitos não são isolados, mas se combinam naturalmente na solução de problemas reais: a lista genérica de itens de um pedido, o cálculo de XP diferenciado por tipo de cliente, as duas assinaturas do método de adicionar item, e a conversão explícita de tipo para acessar o resgate de pontos do Cliente VIP.

Por fim, o uso de exceções customizadas e de uma interface para a regra de desconto evidenciou como a Orientação a Objetos favorece um código mais robusto, extensível e de fácil manutenção — características essenciais para qualquer sistema que, como este, possa vir a crescer com novas regras de negócio no futuro.